

operating system

Introduction to Linux what an Operating System is and how it works.

Operating System

An **Operating System (OS)** is **system software that acts as a bridge between the user and the computer hardware**. It manages system resources like CPU, memory, and storage, and allows us to run applications easily.

Linux is one of the most popular operating systems used in servers

Simple Definition

An Operating System is a program that:

- Controls hardware (CPU, memory, disk, etc.)
- Runs applications (like browser, music player)
- Provides an interface for users (screen, icons, commands)

Real-Life Example

Example 1: Watching a movie

- You open VLC Media Player
- The OS loads the app into memory
- It uses CPU to run the video
- It sends sound to speakers and video to screen

Without OS, VLC cannot work with hardware.

TYPE OF OPERATING SYSTEM

1. Desktop OS
2. Server OS



Desktop Operating System

👉 A **Desktop OS** is designed for **personal use** (one user, daily tasks)

Examples:

- Windows 11
- macOS
- Ubuntu Desktop

What it's used for:

- Watching movies 🎬
- Browsing internet 🌐
- Coding 💻
- Gaming 🎮

Key features:

1. User-friendly GUI
2. Multimedia Support
 - Supports audio, video, graphics
 - Used for movies, games, editing
3. Supports software like:
 - Browsers (Chrome)
 - Media players (VLC)
 - IDEs (VS Code)



Server Operating System

👉 A **Server OS** is designed to **handle multiple users and services continuously**

Server Operating System is an OS designed to run on servers and provide services to multiple users .

👉 Unlike a desktop OS (used by one person), a server OS serves many users, applications, and devices at the same time.

Examples:

- Ubuntu Server
- Windows Server
- Red Hat Enterprise Linux

What it's used for:

- Hosting websites 🌐
- Running databases 🗄️
- Handling APIs / backend apps
- Cloud & DevOps infrastructure

Key features:-+

- High stability (runs 24/7)
- Strong security 🛡️
- Handles many users at once
- command-line (no heavy GUI)
- Many users can access the server at the same time
- High Availability (24/7 Uptime)

what is linux

WHAT IS LINUX :

Linux is an open-source kernel based operating system that manages computer hardware and allows users to run applications. Basically its based on **unix**

HISTORY:

1. Background (Before Linux)

- In the 1970s–80s, the popular OS was Unix
 - Unix was powerful but not free and mostly used by big companies and universities
-

2. Birth of Linux (1991)

- In 1991, Linus Torvalds (a student from Finland) created Linux
- He wanted a free operating system similar to Unix
- He released the first version called Linux Kernel 0.01

Linux is compatible with unix command and software

Linux is not complete an operating system its a kernel

What is kernel ?

A kernel is the core part of an operating system that controls hardware and manages system resources.

👉 It acts as a bridge and it manage communication between applications (software) and hardware (CPU, memory, devices).

⚙️ What Kernel Does (Main Functions)

1. Process Management
 - Runs and controls programs
2. Memory Management
 - Allocates RAM to applications
3. Device Management
 - Controls keyboard, mouse, disk, printer
4. File System Management
 - Handles files and storage

A Linux distribution is a complete operating system made from a collection of software, which includes the Linux kernel, system tools, libraries, and a package management system.

Popular Linux Flavours:

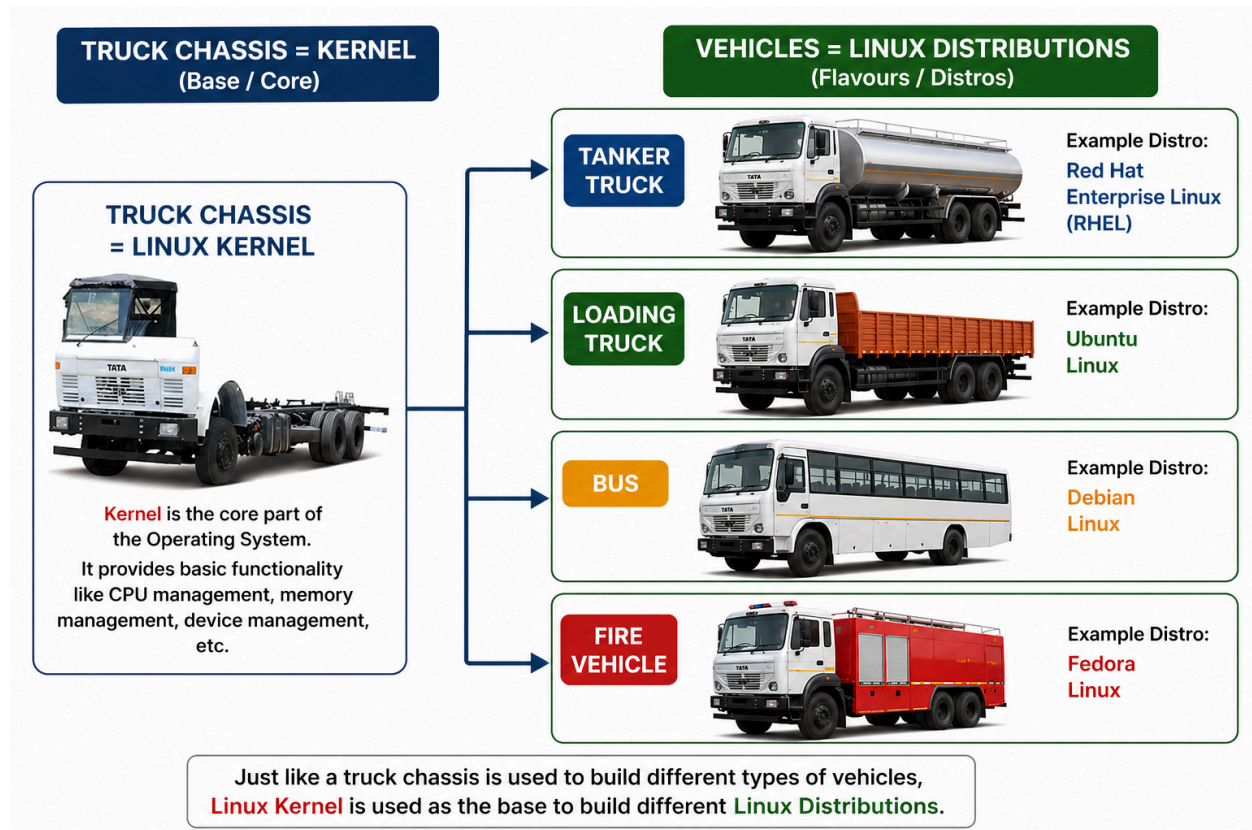
Ubuntu → Easy & beginner-friendly

Debian → Very stable

Fedora → Latest features

Red Hat Enterprise Linux → Enterprise use

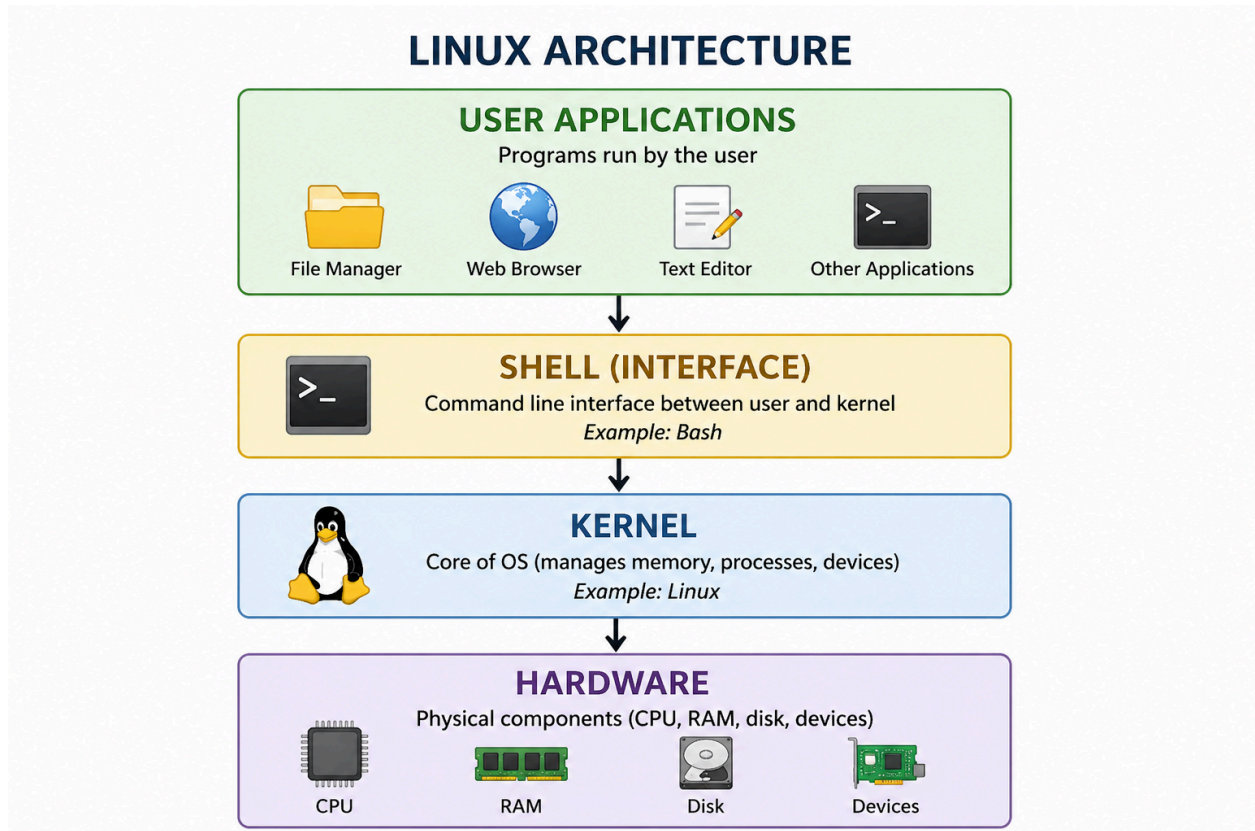
Kali linux → for hacking & high security



Key Features of Linux

- Open Source: Source code is freely available and can be modified.
- Multiuser: Multiple users can access system resources at the same time.
- Multitasking: Can run multiple tasks simultaneously.
- Security: Provides strong security features with regular updates.
- Portability: Can run on various hardware platforms.

Linux Architecture



AWS IAM (Identity and Access Management)

AWS IAM (Identity and Access Management)

IAM is a core service of Amazon Web Services that allows you to **securely control access to AWS resources**.

👉 It answers 2 main questions:

- **Who** can access AWS?
 - **What** actions they can perform?
-



“IAM is a global AWS service used to manage authentication (who you are) and authorization (what you can do) using users, roles, and policies.”

Core Concepts

1. Users (Identity)

- Represents a **person or application**
- Has:
 - Username & password (console)
 - Access key & secret key (CLI/API)

👉 Example: DevOps engineer login

2. Groups

- Collection of users
- Used to **manage permissions easily**

👉 Example:

- DevOps group → EC2, S3 access
 - QA group → Read-only access
-

3. 🤖 Roles

- No username/password
- Provides **temporary access using STS (Security Token Service)**

👉 Used for:

- EC2 instance accessing S3
- Lambda accessing DynamoDB
- Cross-account access

👉 Example:

EC2 role → access S3 without storing keys

4. 📜 Policies (Heart of IAM)

- JSON documents that define permissions
- Attached to users, groups, or roles

👉 Two types:

- **Managed Policies** (AWS or customer-created)
 - **Inline Policies** (directly attached)
-

Example Policy:

```
{  
  "Effect": "Allow",  
  "Action": "s3:ListBucket",  
  "Resource": ""  
}
```

👉 Meaning: Allow listing S3 buckets



Authentication vs Authorization

- **Authentication** → Login (username/password, access key)
- **Authorization** → Permissions (policies)

👉 Example:

- Login success = Authentication
 - Access S3 allowed = Authorization
-



Types of Access

1. Console Access

- Username + Password
- Used for AWS UI login

2. Programmatic Access

- Access Key + Secret Key
- Used in:
 - CLI
 - Terraform
 - Scripts
 - CI/CD tools (like Jenkins)

IAM Policy Types (Definition + Example)

In Amazon Web Services IAM, policies are used to **control access (allow/deny actions)**.

1 Identity-Based Policies

Definition:

Policies **attached to IAM users, groups, or roles** to define what actions they can perform.

Example:

```
{  
  "Effect": "Allow",  
  "Action": "s3:ListBucket",  
  "Resource": "*"   
}
```

Meaning:

- User can **list S3 buckets**

Real Example:

- DevOps user allowed to access S3
-

2 Resource-Based Policies

Definition:

Policies **attached directly to a resource** (like S3 bucket) to define **who can access it**.

Example:

```
{
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::123456789012:user/DevUser"
  },
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::my-bucket/*"
}
```

Meaning:

- DevUser can access this bucket

Real Example:

- Cross-account S3 access
-

3 Permissions Boundaries

Definition:

A policy that **sets the maximum permissions** an IAM user or role can have.

Example:

```
{
  "Effect": "Allow",
  "Action": "s3:*",
  "Resource": "*"
}
```

Meaning:

- User **cannot go beyond S3 access**, even if other policies allow more

💡 Real Example:

- Restrict junior developer to only S3
-

4 Service Control Policies (SCP)

📖 **Definition:**

Policies used in **AWS Organizations** to set **maximum permissions for an account**.

✅ **Example:**

```
{  
  "Effect": "Deny",  
  "Action": "ec2:TerminateInstances",  
  "Resource": "*" }  
}
```

👉 **Meaning:**

- No one in that account can terminate EC2

💡 Real Example:

- Protect production environment
-

5 Resource Control Policies (RCP)

📖 **Definition:**

Policies that **limit what actions can be performed on resources** across accounts.

✅ **Example:**

👉 Block deletion of S3 buckets in organization

👉 **Meaning:**

- Even if user has permission → still cannot delete

💡 Real Example:

- Prevent critical resource deletion
-

6 Access Control Lists (ACLs)

📖 **Definition:**

A **basic permission system** (not JSON) used mainly in S3 to allow access to other accounts.

✅ **Example:**

- Grant READ access to another AWS account

👉 **Meaning:**

- That account can read objects

💡 Real Example:

- Old method for cross-account sharing
-

7 Session Policies

📖 **Definition:**

Policies used to **limit permissions during a temporary session** (like when assuming a role).

✅ **Example:**

👉 Role has full S3 access

👉 Session policy allows only read

👉 **Final:**

- Only read allowed in that session

💡 Real Example:

- Temporary secure access for CLI users